32 Bit Floating-Point Logic Unit (FLU)

Project Documentation and Technical Notes

Project Name: Floating-Point Logic Unit (FLU)
Implementation Language: Verilog HDL
Development Platform: Xilinx Vivado
Data Format: IEEE 754 single-precision floating-point (32-bit)
Design Type: Combinational digital logic
Developer: Ghulam Yaseen Nangraj
Date: October 2026

---

1. Introduction

The Floating-Point Logic Unit (FLU) is an educational digital hardware project designed to perform arithmetic operations on 32-bit floating-point numbers using Verilog HDL.

The project demonstrates how common mathematical operations can be implemented in hardware using combinational logic. Instead of relying on a software processor to calculate results, the FLU uses dedicated Verilog modules to process floating-point operands and produce results according to a selected operation.

The design is intended for learning, simulation, digital logic experimentation, and understanding the internal structure of a basic arithmetic unit.

The project includes separate modules for arithmetic operations, a top-level controller that selects the required result, a decimal-to-floating-point encoder, and a testbench for verification.

2. Purpose of the Project

The primary purpose of this project is to understand how floating-point arithmetic can be implemented in digital hardware.

The project aims to:

- Demonstrate the use of Verilog HDL for arithmetic circuit design.
- Explain the representation of real numbers using the IEEE 754 binary32 format.
- Implement basic arithmetic operations using separate hardware modules.

- Demonstrate how multiple arithmetic units can be connected through a top-level module.
- Show how an operation selector controls which result appears at the output.
- Provide a decimal-input encoding method for preparing floating-point operands.
- Verify the design through simulation and testbench-based checking.
- Provide a foundation for future work involving more accurate arithmetic units, pipelining, and FPGA implementation.

This is an educational implementation rather than a claim of full IEEE 754 compliance or production-level floating-point performance.

3. Project Overview

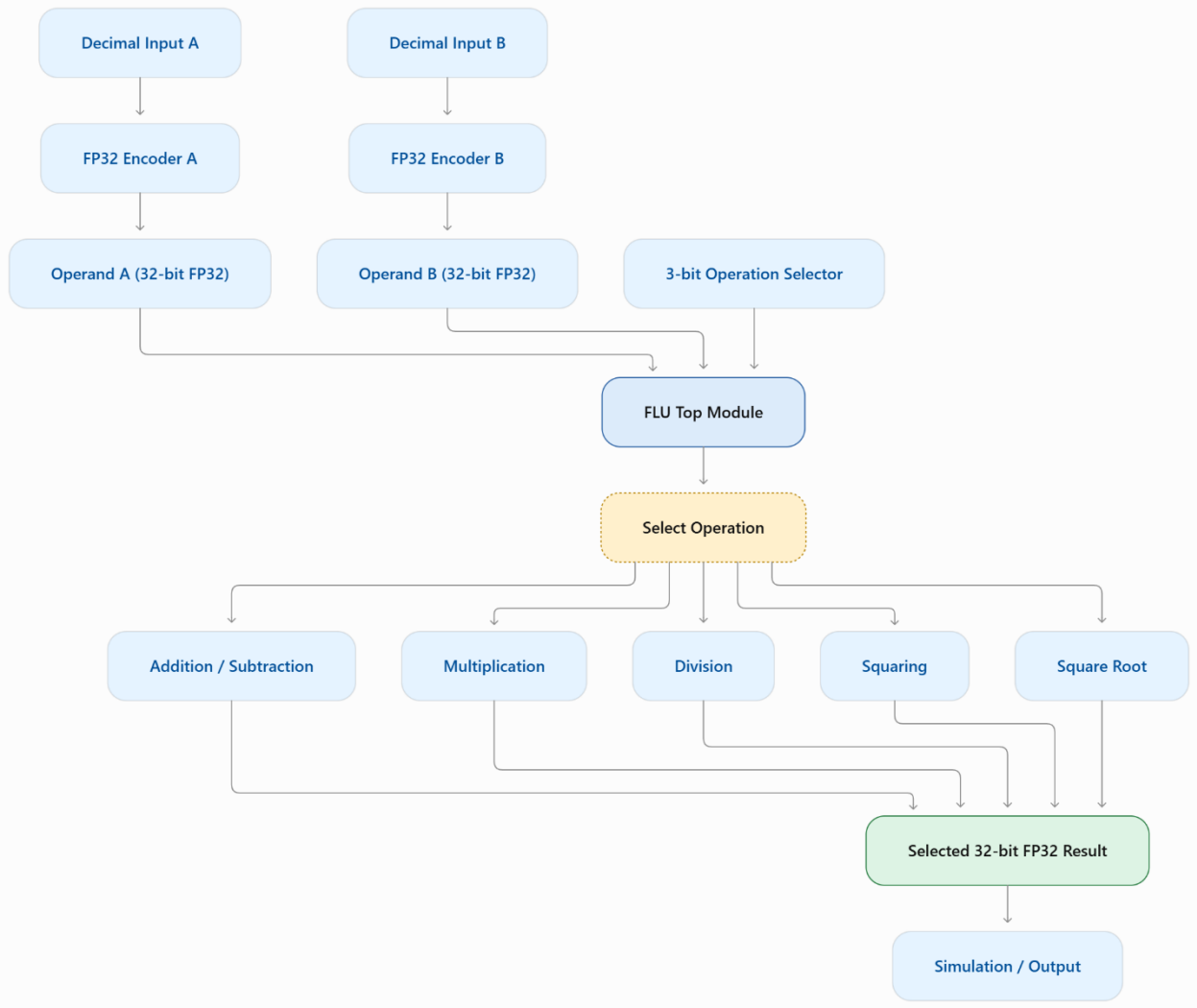
The FLU accepts two floating-point operands, processes them through the selected arithmetic operation, and produces a 32-bit floating-point result.

A separate encoder can convert decimal whole-number and fractional inputs into IEEE 754 binary32 values before they are supplied to the FLU.

The general processing flow is:

1. Provide decimal input values when using the encoder.
2. Convert the decimal values into 32-bit floating-point representations.
3. Supply the encoded operands to the FLU.
4. Select the desired operation using a 3-bit operation code.
5. Allow the corresponding arithmetic logic to calculate the result.
6. Select the appropriate result at the top-level output.
7. Observe the output and any relevant status signals in simulation.

The overall structure can be represented as follows:



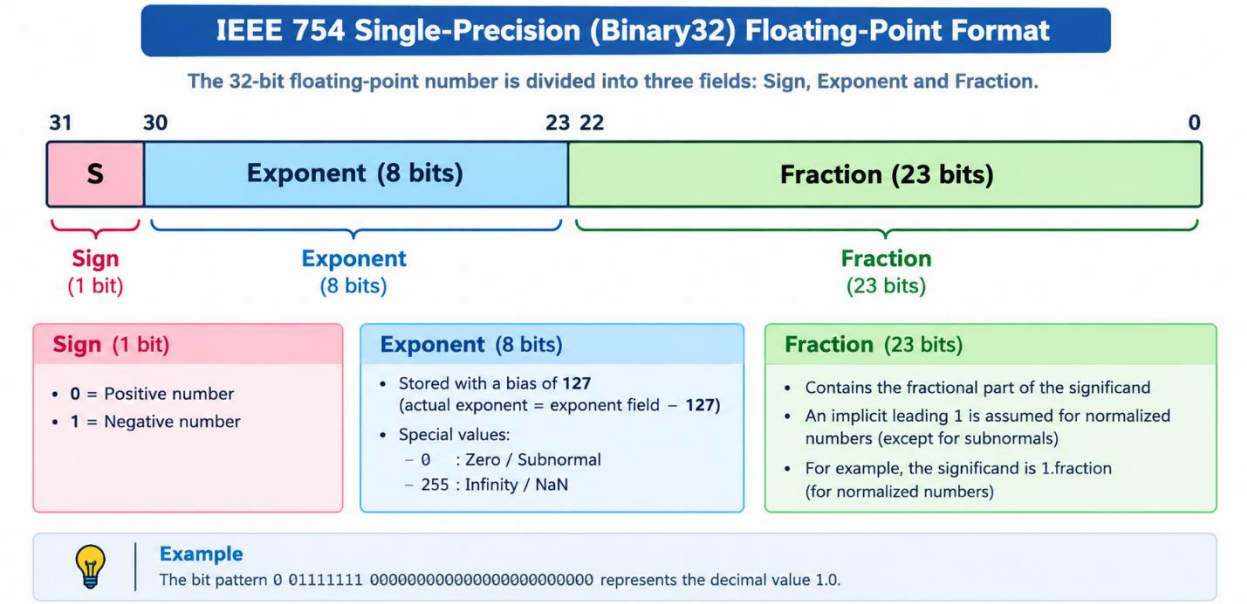
The encoder is a separate input-preparation module. Depending on how the testbench or a future hardware interface is configured, the FLU may also receive floating-point bit patterns directly.

4. IEEE 754 Single-Precision Format

The FLU uses a 32-bit representation commonly known as IEEE 754 single precision or binary32.

A binary32 value consists of three fields:

Field	Bit positions	Width	Purpose
Sign	31	1 bit	Represents positive or negative values
Exponent	30–23	8 bits	Stores the exponent using a bias
Fraction (significand field)	22–0	23 bits	Stores the fractional part of the significand



For normal finite values, the represented number is:

$$(-1)^{S}(1.F)_2 2^{E-127}$$

Where:

- (S) is the sign bit.
- (E) is the stored exponent.
- (F) is the fraction field.
- (127) is the exponent bias for binary32.

- $((1.F)_2)$ is the significand with an implicit leading one.

For example, the decimal number (1.5) has the binary32 representation:

Hexadecimal: 0x3FC00000

Sign: 0

Exponent: 01111111

Fraction: 10000000000000000000000

Its sign is positive, its unbiased exponent is zero, and its significand is (1.1_2), which equals (1.5).

Important: The 32-bit operand is not an ordinary integer representation of the decimal number. It is a bit pattern that encodes a floating-point value.

5. Operations Supported by the FLU

The top-level FLU uses a 3-bit operation selector to choose one of six operations.

Operation code	**Operation**	**Mathematical expression**	**Purpose**
3'b000	Addition	(A+B)	Adds two floating-point operands
3'b001	Subtraction	(A-B)	Subtracts operand B from operand A
3'b010	Multiplication	(Ax B)	Multiplies two floating-point operands
3'b011	Division	(A/B)	Divides operand A by operand B
3'b100	Square	A^2	Multiplies operand A by itself
3'b101	Square root	$\sqrt{A}$	Calculates the square root of operand A

The square operation uses operand A and does not require operand B mathematically. The square-root operation also uses operand A.

The remaining two operation codes, 3'b110 and 3'b111, are reserved unless the design explicitly assigns behavior to them.

5.1 Addition

Addition calculates the sum of two floating-point values.

R = A + B

A floating-point addition unit generally needs to compare exponents, align significands, add or subtract the aligned values as appropriate, normalize the result, and encode it in binary32 format.

The precise handling of these steps depends on the implementation of fp32_add_sub.

Example:

1.5 + 2.25 = 3.75

Expected binary32 result:

0x40700000

5.2 Subtraction

Subtraction calculates the difference between two operands.

R = A - B

The same arithmetic module may implement both addition and subtraction, using a control signal to distinguish between the two operations.

For subtraction, the relative signs and magnitudes of the operands determine whether the significands are effectively added or subtracted.

Example:

3.5 - 1.25 = 2.25

Expected binary32 result:

0x40100000

5.3 Multiplication

Multiplication calculates the product of the two operands.

R=A x B

A typical floating-point multiplier combines the operand signs, adds the unbiased exponents, multiplies the significands, and normalizes the product.

Example:

1.5 x 2.0 = 3.0

Expected binary32 result:

0x40400000

The dedicated module for this operation is fp32_mul.

5.4 Division

Division calculates the quotient of operand A divided by operand B.

R= A/B

A floating-point divider must account for the signs and exponents of both operands and calculate the quotient of their significands. Normalization and appropriate result encoding are also required.

Example:

6.0 / 2.0 = 3.0

Expected binary32 result:

0x40400000

The division module is fp32_div.

The design also provides a divide_by_zero status signal. Its exact behavior should be confirmed from the divider and top-level source code, particularly for zero divided by zero and special-value operands.

5.5 Square

The square operation calculates the square of operand A.

R = A^2 = A x A

It is useful for demonstrating a unary arithmetic operation implemented using floating-point logic.

Example:

1.5 ^ 2 = 2.25

Expected binary32 result:

0x40100000

The dedicated module is fp32_square.

5.6 Square Root

The square-root operation calculates the nonnegative square root of a nonnegative input.

R= $\sqrt{A}$

Square-root hardware is more involved than simply changing the exponent. It must calculate the square root of the significand and handle exponent parity, normalization, and result formatting.

The module fp32_sqrt uses a combinational integer square-root procedure. Its implementation adjusts the exponent, constructs a scaled radicand, and performs iterative digit-by-digit square-root calculations.

Example:
sqrt(1.5) = 1.224744870

Expected binary32 result:

0x40000000

The square-root implementation has documented limitations, including truncation of the calculated significand and flushing positive subnormal inputs to zero. Therefore, its results should be tested against a trusted floating-point reference before claiming full IEEE 754 compliance.

6. Main Modules and Their Responsibilities

The project separates major functions into individual Verilog modules. This modular structure makes the design easier to understand, test, and maintain.

6.1 fp32_encode_comb

Purpose: Converts decimal whole-number and fractional input components into a 32-bit floating-point representation.

The module accepts fields representing the whole part, the fractional part, and the number of fractional decimal digits.

For example, a decimal value of (1.5) can be represented using:

whole = 1

fraction = 5

fractional_digits = 1

The encoder uses a fixed-point intermediate representation with Q40 scaling to construct and normalize the floating-point result.

Its processing includes decimal fraction scaling, normalization, exponent calculation, significand construction, and rounding.

The described implementation uses round-to-nearest, ties-to-even logic. However, its finite intermediate precision means that the encoder should be tested for rounding-sensitive inputs and boundary conditions.

This module prepares decimal test inputs; it is not the arithmetic unit itself.

6.2 fp32_add_sub

Purpose: Performs floating-point addition and subtraction.

The top-level module selects the required behavior based on the operation code. The module's exact internal control interface should be taken from its Verilog declaration.

6.3 fp32_mul

Purpose: Performs floating-point multiplication.

It receives binary32 operands and generates the encoded product.

6.4 fp32_div

Purpose: Performs floating-point division.

It generates the quotient and supports the design's division-by-zero status handling.

6.5 fp32_square

Purpose: Calculates the square of a floating-point operand.

The operation uses operand A as both mathematical inputs.

6.6 fp32_sqrt

Purpose: Calculates the square root of a floating-point operand.

It uses an iterative combinational integer square-root procedure and produces a binary32-formatted result. Its accuracy and exceptional-value handling are limited by the implementation.

6.7 flu_top

Purpose: Integrates the arithmetic modules into one functional unit.

The top-level module receives the operands and operation selector, connects the arithmetic units, selects the appropriate result, and exposes the output and status signals.

This arrangement separates **calculation** from **operation selection**.

6.8 tb_flu_top

Purpose: Verifies the design in simulation.

The testbench supplies operands and operation codes, waits for combinational logic to settle, and checks whether the output matches an expected binary32 value.

A testbench is a simulation component, not necessarily part of the physical FPGA circuit.

7. How the Combinational Design Works

A combinational circuit produces outputs from its current inputs without storing a result for use in a later clock cycle.

The FLU is designed around this principle.

Step 1: Apply the inputs

Two 32-bit operands, A and B, are supplied to the top-level module. These may be produced by the encoder or entered directly as binary32 bit patterns.

Step 2: Select an operation

The operation selector specifies which calculation is required.

For example:

operation = 3'b000; // Addition

operation = 3'b001; // Subtraction

operation = 3'b010; // Multiplication

operation = 3'b011; // Division

operation = 3'b100; // Square

operation = 3'b101; // Square root

These are example assignments for a testbench or other driving logic, not a requirement to place all assignments together in the synthesizable top-level module.

Step 3: Calculate the candidate results

The arithmetic modules receive the operands and calculate their respective outputs according to their implementations.

Unless the RTL explicitly gates or otherwise controls these modules, multiple arithmetic units may calculate candidate results simultaneously.

Step 4: Select the required result

The top-level selection logic routes the result corresponding to the chosen operation to the final output.

For example, when the operation code is 3'b010, the multiplier's output should be selected.

Step 5: Observe the output

After the input and control signals propagate through the combinational logic, the result becomes available.

In simulation, a small delay may be used before checking the output to allow the modeled logic to settle. On an FPGA, the real propagation delay depends on synthesis, placement, routing, and the target device.

No clock is required for the intended combinational calculation. If a future version introduces registers, pipeline stages, or sequential control, that version will have additional timing and clocking requirements.

8. Example Test Cases

The following examples are useful starting points for functional verification. Expected results are shown as 32-bit IEEE 754 binary32 hexadecimal patterns.

Operation	Operand A	Operand B	Expected result
Addition	1.5	2.25	0x40700000
Subtraction	3.5	1.25	0x40100000
Multiplication	1.5	2.0	0x40400000
Division	6.0	2.0	0x40400000
Square	1.5	Not required	0x40100000
Square root	4.0	Not required	0x40000000

These are mathematical reference values, not a guarantee that every current module will pass them. The actual RTL should be simulated and checked before the results are treated as verified.

For the square-root unit, use nonnegative, exactly representable test inputs first. Add edge cases after the basic implementation has been validated.

9. Testbench and Verification Strategy

A well-structured testbench should verify both normal arithmetic and exceptional conditions.

9.1 Basic functional tests

Test simple values that produce exactly representable results, including:

- Zero and one.
- Positive integers.
- Negative operands.
- Exactly representable fractions such as 1.5 and 2.25.
- Addition and subtraction with equal operands.
- Multiplication by zero and one.
- Division by a nonzero operand.
- Squaring positive and negative values.
- Square roots of perfect squares.

9.2 Boundary and special-value tests

When supported by the design, test:

- Positive and negative zero.
- Positive and negative infinity.
- Quiet and signaling NaN bit patterns, if those cases are intentionally handled.
- The smallest normal and subnormal values.
- Overflow and underflow.
- Division by zero and zero divided by zero.
- Negative square-root inputs.
- Rounding near a representable-number boundary.
- All operation codes, including reserved codes.

These cases are particularly important because a result that is correct for ordinary inputs may still be incorrect for special values.

9.3 Comparison methods

For deterministic tests with exact expected results, compare the full 32-bit output against the expected hexadecimal pattern.

For operations where rounding may differ, compare the numerical result against a trusted IEEE 754 reference and define an appropriate error tolerance or ULP-based criterion.

The testbench should report:

- The operation being tested.
- The input operands.
- The expected result.
- The actual result.
- Pass or fail status.

A successful simulation of a limited test set demonstrates that those test cases passed; it does not establish complete arithmetic correctness.

10. Important Design Limitations

The current project is educational in scope. Its behavior should not be assumed to match every requirement of a fully compliant IEEE 754 floating-point unit.

The following limitations deserve particular attention.

10.1 Rounding behavior

Different modules may use different rounding approaches. For example, the encoder uses a Q40 intermediate representation and rounding logic, while the square-root implementation truncates the significand.

This can cause results to differ from a correctly rounded IEEE 754 reference.

10.2 Subnormal numbers

Subnormal numbers represent very small nonzero values that cannot use the normal implicit-leading-one representation.

The described square-root implementation flushes positive subnormal inputs to zero. Other modules may handle subnormals differently.

10.3 Exceptional values

IEEE 754 defines behavior for infinities, NaNs, signed zeros, and invalid arithmetic operations.

Each module should be inspected to determine which cases it supports. A module's ability to recognize an exponent field of all ones does not, by itself, establish complete special-value handling.

10.4 Overflow and underflow

Very large results can overflow the representable range, while very small results can underflow into subnormal values or zero.

The exact output and status behavior depend on each module's implementation.

10.5 Division-by-zero behavior

The divider exposes a divide_by_zero status signal, but the expected output for every zero-divisor case must be verified. In particular, zero divided by zero is different from a nonzero value divided by zero under IEEE 754.

10.6 Square-root accuracy

The current square-root module uses an iterative integer method and truncates the significand. Its output should be checked against a trusted reference for exactness, rounding, exponent handling, and exceptional values.

10.7 Encoder precision

The decimal encoder uses finite fixed-point precision. Some decimal inputs cannot be represented exactly in binary, and intermediate truncation may affect the final bit pattern.

10.8 Timing and hardware resources

Although the design is combinational, its arithmetic paths still have propagation delays. The square-root iteration and other arithmetic logic may produce long critical paths.

Synthesis reports should be reviewed for:

- Maximum combinational path delay.
- LUT and other logic utilization.
- Routing delay.
- Resource usage.
- Timing constraints and timing violations.

A combinational design is not automatically faster or more resource-efficient than a pipelined design.

11. Recommended Project Workflow in Vivado

A typical workflow for this project is:

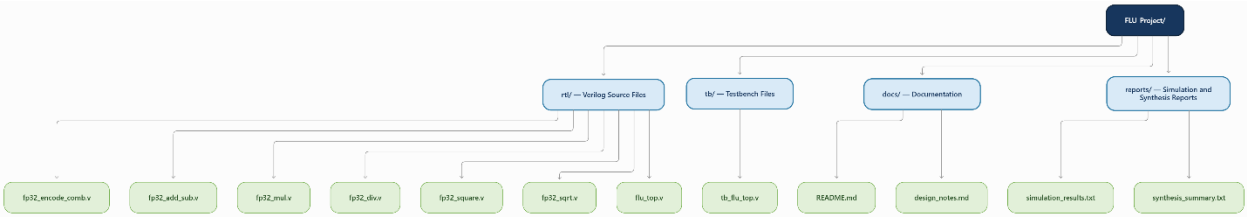
1. **Create the project:** Select the appropriate FPGA device or development board target.
2. **Add the RTL sources:** Include the encoder, arithmetic modules, and top-level module.
3. **Add the testbench:** Include tb_flu_top as a simulation source.
4. **Check module interfaces:** Confirm module names, port widths, signal directions, and operation codes.
5. **Run syntax and elaboration checks:** Resolve compilation errors and incorrect connections.

6. **Run behavioral simulation:** Execute the testbench and inspect the waveform and test results.
7. **Verify arithmetic accuracy:** Compare results with trusted binary32 reference values.
8. **Synthesize the design:** Inspect inferred hardware and synthesis warnings.
9. **Review timing reports:** Determine whether the combinational paths meet the intended timing requirements.
10. **Implement on hardware if required:** Assign physical I/O pins and add constraints appropriate to the target board.
11. **Document verified behavior:** Record supported cases, known limitations, and test results.

If the project is simulation-only, physical implementation and board programming are optional.

12. Suggested Source-File Organization

A clear project structure helps other developers navigate the code.



This is a recommended organization. The actual file names and folder structure may differ in the current Vivado project.

13. Possible Future Improvements

The project can be extended in several directions.

Improve IEEE 754 compliance. Implement consistent handling of subnormal values, signed zeros, infinities, NaNs, overflow, underflow, and rounding across all arithmetic modules.

Improve square-root accuracy. Add appropriate rounding and verify the iterative algorithm against a trusted reference over a wide range of input values.

Strengthen verification. Use automated reference-model comparisons and randomized testing, including boundary values and special cases.

Add status flags. Depending on project requirements, expose invalid-operation, overflow, underflow, and inexact-result flags alongside the existing division-by-zero status.

Optimize timing. Review synthesis reports and consider a pipelined architecture if the target clock frequency requires shorter logic paths.

Improve the input and output interface. Add switches, buttons, display logic, UART, or another interface if the project is intended for physical FPGA demonstration.

Expand supported operations. Additional operations could include absolute value, comparison, negation, reciprocal, or other mathematical functions.

Add automated regression testing. Maintain a repeatable suite of test vectors so that changes to one module do not silently break other operations.

14. Key Terminology

Term	**Meaning**
FLU	Floating-Point Logic Unit
RTL	Register-Transfer Level description of digital hardware
HDL	Hardware Description Language
IEEE 754	Standard for floating-point arithmetic representation and behavior
Binary32	32-bit single-precision floating-point format
Combinational logic	Logic whose outputs depend on current inputs rather than stored state
Significand	The precision-carrying part of a floating-point number
Exponent bias	Constant added to the exponent for its stored representation
Normalization	Adjusting a significand and exponent to the selected representation
Rounding	Selecting a representable result when the exact value cannot be represented
Testbench	Simulation code that drives inputs and checks a design's outputs
Synthesis	Converting RTL into a hardware implementation
FPGA	Field-Programmable Gate Array

15. Conclusion

The Floating-Point Logic Unit is a modular Verilog project that demonstrates the structure of a small floating-point arithmetic system. It combines dedicated arithmetic modules with a top-level operation selector and a separate decimal-to-binary32 encoder.

Its six supported operations—addition, subtraction, multiplication, division, squaring, and square root—provide a useful foundation for studying digital arithmetic, Verilog module integration, combinational logic, and simulation-based verification.

The project's most important educational value is that it makes the relationship between floating-point representation, arithmetic algorithms, hardware architecture, and verification visible.

Before the FLU is used in a performance-critical or standards-sensitive application, every arithmetic module should be verified against a trusted reference, its exceptional-value behavior should be documented, and its timing and synthesis results should be reviewed.

Project summary: The FLU is an educational combinational floating-point arithmetic design built in Verilog HDL, intended to demonstrate how multiple arithmetic operations can be integrated into a single selectable hardware unit.